ENGINEERING FIELD GUIDE · 2026.10

从“刷 B 站”到懂球帝 API 阅读助手

基于 dsh-bili-scroll 的 Python 后端开发方案

> **从用户授权的 APP 请求中核实接口，再用 Python 自建只读适配层。**

方案研究　面向 JavaScript 前端转 Python 后端的开发者　2026-10-24

事实、建议与待验证假设分开标注

01 / FIELD GUIDE

摘要

一句话结论

开发懂球帝阅读助手，先从用户授权的 APP 正常请求中确认接口契约，再用 Python 自建只读适配层，逐步加入判重、日报与 MCP 工具。

这份方案的结论

> **先核实接口契约，再设计后端。** 哔哩哔哩有官方开放平台；参照仓库内置的 B 站 MCP 则来自开源社区项目，不应误称为 B 站官方 MCP。懂球帝没有本报告可核验的官方 MCP 文档，因此按“从用户授权的 APP 正常请求中观察接口、自建只读 Python 适配层”设计。私有客户端接口可能变更，不等同于稳定公开 API。

本报告将目标范围定义为个人阅读辅助：用用户自己的懂球帝账号和设备，在授权范围内读取指定信息流或文章，保存文章标识、阅读状态和本地摘要。接口观察从用户在 APP 中正常操作开始；实现阶段只发出本轮已确认、被允许且只读的请求。项目不设计虚假浏览量、模拟观看时长、自动点赞、评论、签到或领取平台奖励。正常打开内容仍可能被平台记录为浏览事件。

报告基于 dsh-bili-scroll 的公开 main 分支代码与文档，并查阅哔哩哔哩开放平台、Python HTTPX 与 MCP 官方资料。按本次公开资料核对，未找到懂球帝官方 MCP 服务入口或面向本项目的个人客户端 API 文档；这是一条有范围的检索结果，不是对未公开服务作绝对否定。懂球帝接口字段、鉴权、分页和访问规则均待在目标设备与授权账号上验证。代码和公开资料核对日期为 2026-10-24。

本次核对的 package.json 显示版本 2.1.0，而 README 的部分安装说明仍写 2.0.2；报告因此依照 main 分支代码分析职责，不把 README 中的旧安装命令当作当前发布指引。

目录

摘要 2

第一幕｜先拆开一个已经跑起来的项目 4

第二幕｜懂球帝没有可用 MCP：自己建立客户端接口适配层 6

第三幕｜把它变成 Python 初学者能维护的项目 10

工具箱｜启动编码前要能回答的问题 14

参考资料 15

第一幕｜先拆开一个已经跑起来的项目

1. 它不是一个 Python 脚本

dsh-bili-scroll 是 DSH 插件，代码分成宿主侧与平台侧。宿主侧是 Node.js：插件入口 index.js 负责注册能力、配置、HTTP 接口与任务启动；client.js 提供 DSH 设置页。平台侧是 Python MCP 服务，承接 B 站数据能力。lib 里放 MCP stdio 客户端、状态目录、扫码登录等共用模块；prompts/duty.md 描述一轮任务的目标与边界；vendor/bilibili-mcp 内置上游 Python 项目。仓库说明；入口 index.js；设置页 client.js



初学者可以把它想成“前台 + 后厨”：Node 插件是前台，负责用户能看到的设置和什么时候开工；Python 服务是后厨，负责平台相关的数据操作。两边用 MCP 工具契约交接。MCP 工具是带名字、输入字段和返回结果的函数接口，类似给 AI 使用的 API。MCP Python SDK

2. 可以沿用的是边界，不是 B 站接口

这里要把“开放平台”和“MCP 服务”分开看。哔哩哔哩官方开放平台提供账号授权、用户管理、视频管理、数据开放等 API 能力，但具体能力要按平台文档和授权条件申请；参照项目内置的 `vendor/bilibili-mcp` 是第三方开源 MCP 服务，把平台或客户端 API 包装成 AI 可调用工具。MCP 是调用接口的协议层，不代表目标平台自己运营了 MCP 服务。哔哩哔哩开放平台文档；参照仓库 README

懂球帝这边，报告按你的项目设定：没有可用的官方 MCP 服务，因此由项目自己实现懂球帝客户端 API 适配器；公开资料检索也没有找到能直接照着调用的官方个人 API 契约。要把请求信息从懂球帝 APP 正常使用过程中采集、整理成可维护的接口契约，而不是把 B 站接口或参数照搬过来。

参照项目里的部分	懂球帝项目怎样处理	判断
Node 插件入口与 DSH 设置页	如果仍以 DSH 为宿主，保留一层薄 JS 插件	可沿用结构
Python MCP 工具边界	定义读取信息流、读取文章、读取运行状态、生成日报等窄工具	可沿用模式
B 站搜索、字幕与账号凭证	不移植；从懂球帝 APP 正常请求样本建立自己的接口契约	平台专属
duty.md 的任务章程	保留阅读范围、输出格式与“文章内容不是指令”的约束	可改写

参照项目里的部分	懂球帝项目怎样处理	判断
本地状态、日志与日报	用户数据独立于安装包；SQLite 保存判重和运行状态	可沿用并简化
工具白名单与专用 profile	只注册实际需要的读取工具；不给任务随意 shell、文件系统或联网能力	应沿用原则

仓库根目录的 package.json 将插件登记为 ESM 包，入口是 index.js，client.js 提供客户端设置页；依赖声明里也能看出 DSH 宿主与 Node 运行时要求。package.json 这说明 Python 是它的一个业务执行层，而不是整个插件的唯一运行时。

3. 一轮任务实际经过哪些环节

README 和入口注释描述的主流程是：设置关键词和每日条数；定时器或用户手动启动一轮；启动 headless DSH 会话；把当天设置和 duty.md 交给会话；会话调用已注册的平台工具；本地保存判重状态和日报。运行说明；任务章程

Python MCP 服务通过标准输入输出（stdio）与 Node 侧通信。参照项目的客户端会先初始化 MCP，再逐行传 JSON-RPC 请求；它同时收集错误输出并处理子进程退出。MCP 客户端实现 因此，Python 进程的标准输出必须留给协议数据，普通日志写到标准错误。把 print 调试信息混进 stdout，会破坏消息流。

这里还有一个关键的安全教训：提示词说“不要做某事”只是规矩说明；真正限制能力，要靠 MCP 工具白名单和宿主 profile。参照仓库按环境变量白名单注册工具，也在写入型函数入口再次检查是否允许调用。Python MCP 工具实现 迁移时应保留这种“提示词讲规则、程序再设闸”的双层设计。

4. 用户数据为何不应放进安装包

参照项目把配置、凭证、日志、报告和运行状态放在插件自己的用户目录，并将需要凭证路径的 Python 脚本复制到该目录运行。原因很直接：插件包可能只读，也会随升级被替换。状态目录实现 懂球帝版本可以用同样原则，把可复现代码留在仓库，把运行数据放进 `.data/` 或宿主提供的用户目录。若已确认的只读接口要求身份凭证，先确认平台允许的凭证来源与使用方法；只有明确获授权的凭证才由客户端本机注入，并从样本、源码、日志中排除。不能因为看到了 APP 请求头，就默认允许提取或重放登录秘密。

5. 开源复用也要看每一层的许可证

参照仓库本体采用 MIT；vendor/bilibili-mcp 也有单独的 LICENSE 和署名信息。NOTICE.md 如果只借鉴架构，按自己的代码与许可证组织即可；如果实际复制或分发代码，应保留对应组件要求的版权行和许可证文本，不能只看仓库最外层的 LICENSE。

第二幕｜懂球帝没有可用 MCP：自己建立客户端接口适配层

5. 先把目标说清楚

本报告推荐的最小产品是“懂球帝阅读助手”，而不是刷平台指标的机器人：

- 用户手动在 APP 中登录；程序不接管密码或验证码。有授权的接口凭证只能按确认过的来源本机配置，不能写进源码、样本文件或日志。
- 一轮只从选定的信息流读取有限条内容。
- 对文章记录接口实际返回的稳定标识、标题、来源和正文；记录本地判重状态。
- 可选生成本地 Markdown 日报，指出哪些内容读到了、哪些没有读到。
- 工具只提供读取和本地记录能力，不注册评论、点赞、签到、奖励或通用请求接口。

实现路线不是预先写一个猜测的 URL，而是先观察 APP 在用户正常操作时发出的请求，再把观察结果整理成自己的接口契约。每一项字段都要有样本证据：真实请求方法和路径、必需参数、认证要求、响应结构、分页字段和错误码。接口未被观察到之前，不填猜测值、不编默认值；登录态、字段或分页契约缺失时，流程明确失败并停止。

6. 从 APP 的正常请求建立一份接口契约

接口发现阶段仍由用户在自己的设备和账号里操作 APP。通过经允许的调试代理或网络检查工具，记录“打开信息流”“翻到下一页”“进入文章”这几种明确动作对应的请求与响应。先看清一个接口，再写代码。

观察时用一张记录表固定接口证据：

记录项	记录什么	为什么要记
用户动作与时间	哪个页面动作触发请求、APP 版本、采集日期	APP 升级后能比较契约变化
请求定位	域名、路径、HTTP 方法、查询参数、请求体字段	还原真实调用，不凭页面名称猜路径
身份与权限	是否需要登录、认证头存在与否、所需账号范围	凭证只本机读取，敏感值不进入文档与样本
响应契约	状态码、内容类型、JSON 字段类型、必需字段	后续解析器严格按已观察结构工作
分页与错误	下一页标记如何返回、结束条件、鉴权/限流/服务错误	没有明确结束标记就不能假定已读完
证据文件	删除敏感值后的请求/响应样本与来源说明	可离线验证解析器，也方便发现接口变化

重复观察同一动作，并改变一项输入（例如文章 ID），对照请求差异。字段须经样本确认后才写入 `schemas.py`；分页只按真实结束标记判断。

> **抓不到请求时，不要绕过客户端的传输保护。** 如果正常授权的调试方式看不到请求，或 APP 明确拒绝代理/重放，就先停止接口发现，取得平台许可或正式接口材料后再继续。不要关闭 TLS 证书校验、破解证书绑定、绕过登录验证、轮换账号或模拟业务写入。

数据和账号权限只使用本人控制或已获授权的账号。先核对平台规则与使用范围；不把 Cookie、Token、个人标识或完整请求头写到 Git、报告、测试夹具和普通日志。报告样本使用脱敏后的字段结构，不保存认证秘密。

7. 推荐架构：Python 适配器连接懂球帝客户端 API

以下架构是开发方案，不代表接口已经实测。当前第一步先读取配置中的球队 ID、初始化 SQLite；完成授权和球队名单契约核验后，才实现第一条只读请求。名单响应解析通过后，球员 ID 写入数据库。CLI 稳定后再加 MCP，MCP 稳定后再加 DSH。



球队 ID 只在 `config.toml` 中配置。球员 ID 不手工写入配置：未来球队名单请求和响应校验成功后，由流程从响应中提取，再通过单个数据库事务更新该队名单。当前 API 客户端、业务方法、认证和响应 schema 都保持占位，不会发出网络请求。

组件的职责要清楚：

模块	做什么	初版边界
config.toml / config.py	保存并读取球队 ID 与 SQLite 文件位置	球队 ID 显式配置；缺字段或类型错误就停止
api/auth.py	后续读取经确认且获准使用的认证来源	当前占位；不能根据旧抓包缺少 Cookie 就断言无需认证

模块	做什么	初版边界
api/client.py	后续发送经过授权和契约核验的 HTTP 请求	当前占位；不包含可执行请求
api/dqd_api.py	后续添加经过核验的球队名单只读方法	当前占位；球队名单是第一条候选请求
api/schemas.py	后续校验脱敏样本所确认的响应字段	当前占位；不提前假定球员 ID 字段路径
workflow.py	后续编排读取球队名单、校验并保存球员 ID	仅在请求和解析都成功后更新数据库
models.py	后续定义项目内部数据结构	字段须来自已确认响应，不复制未核实字段
repository.py	初始化 SQLite 并保存球队/球员 ID 关联	当前建立 `teams`、`players`、`team_players`；名单更新在事务中完成
report.py	输出 Markdown 日报	只写程序实际读到的内容
mcp_server.py	后续将少量 Python 函数暴露给 MCP 客户端/DSH	不暴露任意 URL、通用请求或写入工具
prompts/duty.md	后续控制摘要口径、内容筛选和输出语气	不能替代代码权限控制
plugin/	可选的 DSH JS 宿主层	只有 CLI 与 MCP 稳定后再做

8. 运行流程与状态机

单轮 API 阅读流程



第三幕｜把它变成 Python 初学者能维护的项目

9. 从前端 JS 转到 Python：先迁移心智模型

前端常见概念	Python 里对应的东西	需要留心
JS object	dict 或 dataclass	dict 可随时增删键；关键业务数据适合用固定字段模型
JS Array	list	列表可以保存一批 Article
null / undefined	None / 缺少字段	两者并不完全等价；必填数据缺失时应显式报错
function	def	函数用缩进形成代码块
fetch / Promise	HTTPX 同步客户端	先按顺序发一条列表请求和一条详情请求，不急着学 async
npm 包与 package.json	Python 包与 pyproject.toml	Python 项目用虚拟环境隔离依赖
try/catch	try/except	捕获具体异常，并把错误上下文记下来
localStorage/文件状态	SQLite	判重需要稳定键、唯一约束与明确状态

Python 包管理的第一件事是创建项目虚拟环境。它让该项目的解释器和依赖独立于系统 Python；官方文档也明确指出环境不应提交到 Git，迁移机器时应重新创建。Python venv 文档 当前阶段用 Python 标准库 `tomllib` 读取球队配置、用 `sqlite3` 建立本地数据库；接口核验前不需要 HTTPX。通过授权和契约闸门后，再加入 HTTPX 并先实现一条同步只读请求；只有后续需要并发时才研究 async。HTTPX 官方文档 MCP Python SDK 的版本选择也留到 CLI 稳定、进入 MCP 阶段时再确认。SDK 当前 README；参照项目 Python 依赖

当前开发顺序是：激活 `dqd` Conda 环境；在 `config.toml` 显式填写球队 ID 与数据库路径；运行 `python -m dqd_scroll db-init --config config.toml` 建表；然后在授权设备上重新核验球队名单接口和响应。契约确认后再添加 HTTPX，实现第一条球队名单只读请求，把经过校验的球员 ID 写进 SQLite。CLI 流程稳定后才加入 MCP，再考虑 DSH。MCP 是集成边界，不是学习 Python 的前置条件。

```
def require_value(name: str, value: str) -> str:
    # API 契约或授权配置缺失时立即停止，避免程序猜测请求目标。
    if not value:
        raise ValueError(f'{name} is required');
    return value;
```

这段示意代码没有替缺失配置塞默认值。API 地址、认证来源、契约版本和数据目录都应来自明确配置；直到在目标设备上核实之前，不要把示例路径、参数名或字段名写成懂球帝的真实值。

发 HTTP 请求时可以先把“获取响应”和“解析响应”分成两步：

```
import httpx;

def request_json(client: httpx.Client, endpoint: str) -> object:
    if not endpoint:
        raise ValueError('endpoint is required');
    response = client.get(endpoint);
    response.raise_for_status();
    return response.json();
```

这里的 endpoint 必须来自已观察并确认的接口契约，代码没有写入懂球帝真实 URL。raise_for_status() 会把 HTTP 错误显式暴露出来；下一步仍要由 schemas.py 检查响应字段，不能把 HTTP 200 当成 JSON 一定符合预期。

10. 目录布局：先少而清楚

```
dqd-scroll/
├── config.toml                # 球队 ID 和 SQLite 文件路径
├── pyproject.toml
├── README.md
├── src/
│   └── dqd_scroll/
│       ├── config.py
│       ├── models.py
│       ├── api/                # 授权/契约确认前保持占位
│       ├── repository.py
│       ├── workflow.py
│       ├── report.py
│       ├── mcp_server.py       # CLI 稳定前保持占位
│       └── cli.py
├── docs/
│   └── api-contract.md
├── fixtures/
│   └── README.md
├── prompts/
│   └── duty.md
├── plugin/                     # DSH 稳定前保持占位
└── .gitignore
```

当前可执行部分只有 CLI 的状态查看、配置读取和 SQLite 初始化。api/、MCP 和 DSH 入口保持占位；docs/api-contract.md 记录旧 Postman 抓包线索和待核验项，不能当成可执行契约。夹具只保存已脱敏且经确认的响应结构，绝不包含 Cookie、Token 或个人数据。

SQLite 适合单机 PoC：不必先部署数据库服务，标准库 sqlite3 就能保存本地状态。Python sqlite3 文档 当前初始化三张最小表：teams(team_id) 登记配置球队，players(person_id) 保存抓到的球员 ID，team_players(team_id, person_id) 保存球队名单关系并设联合主键。当前没有文章或运行记录表，因为相应接口和字段尚未确认；等阅读流程及稳定文章 ID 有证据后再增加。

11. 什么情况下才接 MCP 与 DSH

Python CLI 稳定后，mcp_server.py 才把有限的能力暴露成工具。例如：

工具名	输入	返回
dqd_check_api	已配置的接口契约标识	明确的连通、鉴权或契约错误
dqd_read_feed	已确认的信息流类型、条数上限	带稳定 ID、标题、来源的文章列表
dqd_read_article	已确认的文章 ID	正文文本、来源、读取状态
dqd_run_reading	一轮显式配置	本轮摘要、成功/跳过/失败条数、报告路径
dqd_run_status	运行 ID	当前状态和最近错误

> 不要给模型一个“请求任意网址”“传入任意请求头”或“执行任意 HTTP 方法”的大工具。MCP 工具只能调用已确认的只读 API 方法，认证值由本地客户端注入并从日志中清除。文章正文只是外部内容，不能当成新的系统指令。

参照项目的 MCP Python SDK 依赖锁在 1.x，是因为它仍使用旧的 FastMCP API；当前官方 Python SDK 主线展示的是 MCPServer，并要求 Python 3.11+。因此，新项目要么明确沿用参照代码对应的 SDK 主版本并锁定依赖，要么采用新版 API 后核对 DSH 侧 MCP 客户端的协议兼容性；不要边写边升级。当前 Python SDK；SDK 迁移说明

12. 分阶段实现与验收

阶段	目标	通过条件
0. 项目基础	配置 `team_id`，初始化 SQLite	`db-init` 成功；三个表存在；无网络访问
1. 授权与名单观察	在用户正常操作 APP 时重新捕获球队名单请求和响应	授权边界、APP 版本、采集时间明确；样本已脱敏
2. 名单契约 PoC	核对路径、参数、认证与球员 ID 字段	每项字段有当前样本支撑；没有猜测 URL 或字段路径
3. 第一条只读请求	用 HTTPX 读取球队名单并严格校验响应	明确失败时不写入；成功时将 person ID 写入 SQLite
4. CLI 流程	配置检查、抓取、校验、更新球队名单	名单更新为一个事务；重复运行结果可解释
5. MCP 接入	CLI 稳定后暴露受限只读工具	工具范围窄；错误可见；stdout 只发协议消息
6. DSH 插件与调度	MCP 稳定后再做设置页和定时	目标可见；用户能暂停；限流与认证错误不会被吞掉

现在的下一步

项目基础可以先在本机完成：检查 `config.toml` 中的目标 `team_id`，运行 `python -m dqd_scroll db-init --config config.toml`。这一步只创建 `teams`、`players`、`team_players`，不请求懂球帝。数据库就绪后，下一步不是猜接口或提前写请求，而是在授权设备上重新抓取球队名单动作，核实请求和响应，再决定是否实现这条只读请求。名单接口选作第一条，是因为球员 ID 要由实际球队数据抓取后写入，不应该预先硬编码。

每阶段都可以先用单条数据验收，不需要先覆盖整个 APP。第一条接口选球队名单：用脱敏 JSON 样本确认球员 ID 字段和类型，再由授权账号做一次低频只读验证；缺 ID 或字段类型变化时，不写入 SQLite。

工具箱｜启动编码前要能回答的问题

13. 第一轮接口检查清单

1. 球队名单样本是否来自本人正常使用懂球帝 APP 的授权账号？
2. APP 版本、采集日期、目标球队、用户动作、请求域名与路径是否都有记录？
3. 请求是否依赖登录态、Cookie、Token 或设备上下文？认证结论是否有正反样本支持？
4. 球员 ID 在名单响应中的字段路径和类型是什么？缺失或重复时是否停止更新？
5. 名单是否分页或按位置/赛季分组？完整名单的停止条件是否有样本证明？
6. 平台规则是否允许这类低频、本地、只读访问？请求频率限制是什么？
7. 球队 ID 是否只从配置读取，球员 ID 是否只在名单抓取并校验成功后写入 SQLite？在这些问题未答清前，不启用定时、并发或写入平台操作。

14. 竞争假说与报告边界

懂球帝客户端 API 的可用性有两种需要实测区分的解释。假说 A：APP 正常操作产生的请求在用户授权范围内可以被记录和重复调用，响应契约稳定到足以解析；假说 B：请求依赖短期登录态、设备上下文、服务端策略或随版本变化，独立客户端无法稳定重现。第一条脱敏样本只能证明“某次看到过响应”，不能证明持续可用；需要对同一只读动作重复采样、对照 APP 版本并核对平台规则。

本报告没有目标设备的实时请求样本、当前鉴权方式、API 条款或接口稳定性证据，因此不能承诺某个懂球帝客户端接口可长期调用。主要风险是接口契约变化、授权过期、限流策略、用户数据被写入日志或平台规则不允许独立重放。API 适配器应集中管理并严格校验响应；认证失败、字段变更或未知状态都应立即停下并记录脱敏诊断，而不是沿用旧字段继续读。

通过客户端接口读取可能产生与浏览相关的服务端事件；本项目不能也不应伪造阅读行为。内容摘要应尽量短并保留标题、来源链接，不把整篇文章重新发布。账号秘密、原始请求头和个人标识不进入仓库；本地数据库也应由用户控制，分享诊断信息前先脱敏。

15. 读开源项目时的四个问题

以后看到任何“帮我自动化某个平台”的项目，先问：

- **谁在控制流程？** 插件、CLI、调度器还是 AI 模型？
- **平台能力从哪里来？** 是官方开放 API、社区 MCP 包装，还是自己维护的客户端 API 适配器？
- **状态和秘密放在哪里？** 配置、登录状态、判重表、日志与报告是否分开？
- **什么动作会产生外部副作用？** 权限是否由程序工具层限制，而不只写在提示词里？

能答清这四个问题，才开始读单个函数。此时你已经知道项目的“骨架”在哪里；剩下的就是把一个真实接口接入一个边界清楚的 Python 函数。

参考资料

- [S1] dsh-bili-scroll 仓库与 README
- [S2] 插件入口与配置：index.js
- [S3] 设置页：client.js
- [S4] 插件元数据：package.json
- [S5] 用户状态目录：lib/state.mjs
- [S6] MCP stdio 客户端：lib/mcp.mjs
- [S7] 一轮任务章程：prompts/duty.md
- [S8] 内置 Python MCP 服务
- [S9] 参照项目 Python 依赖约束
- [S10] 哔哩哔哩开放平台文档中心
- [S11] 哔哩哔哩开放平台开发者服务协议
- [S12] HTTPX 官方文档
- [S13] MCP 官方 Python SDK
- [S14] MCP Python SDK 迁移说明
- [S15] Python 虚拟环境文档
- [S16] Python sqlite3 文档
- [S17] 参照仓库第三方代码与署名说明

来源与边界：S1–S9 用于描述参照项目；S10–S11 证明哔哩哔哩提供官方开放平台 API 能力，S1 与 S8 显示参照仓库使用了内置的开源 MCP 服务；S12–S16 用于解释拟采用的工具与 Python 概念。懂球帝没有本报告可核验的官方 MCP/个人 API 契约；接口与访问条件仍需由用户在授权设备和账号上确认。本报告是开发方案，不是已完成的客户端实现或接口验收报告。